

Dynamic Memory and Cache

David Marchant
Based on slides by Troels Henriksen

2026-09-16

Arrays

Memory hierarchies

Cache organisation and operation

Cache performance

Arrays

Definition

An array is a multidimensional sequence of objects of the *same type and size*.

- Arrays often used to represent mathematical objects such as *vectors*, *matrices*, and *tensors*.
- Probably the most common data structure for scientific data.
- The arrays we will cover in this lab (and course) are
 - ▶ **Regular:** all “rows” of a multi-dimensional array have the same size.
 - ▶ **Homogeneous:** all elements have the same type.

Regular arrays

In Python and F#, we can have lists of lists with irregular shapes:

```
>>> a = [[1,2,3], [4]]
```

```
>>> len(a)
```

```
2
```

```
>>> len(a[0])
```

```
3
```

```
>>> len(a[1])
```

```
1
```

- Such structures are **irregular**, and outside today's topic.
- What we will discuss is more similar to NumPy arrays.

So what's wrong with C arrays

We can declare an $n \times m$ array as

```
double A[n][m];
```

And then we can index it with for example `A[1][2]`. Easy!

So what's wrong with C arrays

We can declare an $n \times m$ array as

```
double A[n][m];
```

And then we can index it with for example `A[1][2]`. Easy!

But there are many problems with built-in arrays:

- They decay to pointers in many situations.
- They cannot be passed to a function without losing their size.
- They cannot be returned from a function at all.
- **They are not values!**

Let's build our own arrays

- C as a *language* does not have useful dynamic arrays.
- But C does have useful support for *dynamic memory allocation*.
- So let's build our own arrays!

Constructing a dynamic array in C

- Use `malloc()` or `calloc()` to obtain a block of memory with room for enough *bytes* to fit the array we need.
- We can view these functions as allocating an “array of bytes”, which we can then *interpret* as arrays of some other type.

Let's build our own arrays

- C as a *language* does not have useful dynamic arrays.
- But C does have useful support for *dynamic memory allocation*.
- So let's build our own arrays!

Constructing a dynamic array in C

- Use `malloc()` or `calloc()` to obtain a block of memory with room for enough *bytes* to fit the array we need.
- We can view these functions as allocating an “array of bytes”, which we can then *interpret* as arrays of some other type.

Questions:

- **How much memory do we allocate?** An x -element array needs $x * \text{sizeof}(t)$ bytes, where t is the element type (`int`, `double`, etc).

Let's build our own arrays

- C as a *language* does not have useful dynamic arrays.
- But C does have useful support for *dynamic memory allocation*.
- So let's build our own arrays!

Constructing a dynamic array in C

- Use `malloc()` or `calloc()` to obtain a block of memory with room for enough *bytes* to fit the array we need.
- We can view these functions as allocating an “array of bytes”, which we can then *interpret* as arrays of some other type.

Questions:

- **How much memory do we allocate?** An x -element array needs $x * \text{sizeof}(t)$ bytes, where t is the element type (`int`, `double`, etc).
- **How do we lay out the array in memory?** That's a more open question...

Different allocating functions

malloc

Allocates a block of memory of a given size. Accessing uninitialised memory is undefined behaviour (E.g. don't do it)

calloc

Exactly the same as malloc, except that the memory is initialised the zero

realloc

Takes an already allocated block of memory and resizes it. No new initialisation takes place (both a blessing and a curse)

Note that use of realloc is often discouraged unless managed correctly, but its worth mentioning here as you will see it online.

What you take, you must give back

- Allocating memory means you claim it, no one else can use it.
- *Eventually* the OS will step in and clear it up for you, but depending on that is seen as **VERY BAD** practice.
- Submitting code that does this is seen as something to be penalised.

What you take, you must give back

- Allocating memory means you claim it, no one else can use it.
- *Eventually* the OS will step in and clear it up for you, but depending on that is seen as **VERY BAD** practice.
- Submitting code that does this is seen as something to be penalised.
- Always always always, free any allocated memory
- use the `free(ptr)` command to free allocated memory, as pointed to by the given pointer
- You can test your program using `valgrind`

Why discourage realloc

- Consider the following code:
- `int *ptr = malloc(12);`

Why discourage realloc

- Consider the following code:
- `int *ptr = malloc(12);`
- If this fails then no memory is allocated. The pointer returned by `malloc` will be `NULL`, so we can detect this, but don't need to free anything.
- But now consider if the above was followed by:
- `ptr = realloc(ptr, 16);`
- If this fails then no memory is allocated and the pointer is again returned as `NULL`. But now we've lost the pointer to that first `malloc`, and that still needs free'd.

The idea

```
int* arr = malloc(12); // reserve 12 bytes
```

- Suppose `malloc()` returns the address 1000.
- When we do `arr[i]`, C computes the address $1000 + i \times \text{sizeof}(\text{int})$ and reads an `int` (four bytes) from that address.
 - ▶ `&arr[0]:1000`
 - ▶ `&arr[1]:1004`
 - ▶ `&arr[2]:1008`
- (Recall that `&x` means “the address of `x`”.)

One-dimensional arrays in C

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    int size = 10;
    int *arr = malloc(size * sizeof(int));

    printf("&arr:_%p\n", (void*)&arr);
    printf("arr:_%p\n", (void*)arr);

    for (int i = 0; i < size; i++) {
        arr[i] = i*2;
        printf("&arr[%d]:_%p_", i, (void*)&arr[i]);
        printf("arr[%d]:_%d\n", i, arr[i]);
    }

    free(arr);
}
```

```
$ gcc 1darray.c -o 1darray
$ ./1darray
&arr: 0x7ffee169ba80
arr: 0x1bb42a0
&arr[0]: 0x1bb42a0 arr[0]: 0
&arr[1]: 0x1bb42a4 arr[1]: 2
&arr[2]: 0x1bb42a8 arr[2]: 4
&arr[3]: 0x1bb42ac arr[3]: 6
&arr[4]: 0x1bb42b0 arr[4]: 8
&arr[5]: 0x1bb42b4 arr[5]: 10
&arr[6]: 0x1bb42b8 arr[6]: 12
&arr[7]: 0x1bb42bc arr[7]: 14
&arr[8]: 0x1bb42c0 arr[8]: 16
&arr[9]: 0x1bb42c4 arr[9]: 18
```

Multi-dimensional arrays

- Machines (and C) provide a *one-dimensional memory (or index) space*.
- When we want multi-dimensional arrays (and we do!) we need to specify a *mapping* between our desired multi-dimensional space and the machine's single-dimensional space.

This is an *index function*.

Index functions

An index function maps a d -dimensional index to a single-dimensional index.

The type of index functions

$$I : \mathbb{N}^d \rightarrow \mathbb{N}$$

Index functions are not necessarily literal C functions, but a *conceptual* description of how the array is laid out in memory.

Index functions

An index function maps a d -dimensional index to a single-dimensional index.

The type of index functions

$$I : \mathbb{N}^d \rightarrow \mathbb{N}$$

Index functions are not necessarily literal C functions, but a *conceptual* description of how the array is laid out in memory.

Inverse index functions

$$I^{-1} : \mathbb{N} \rightarrow \mathbb{N}^d$$

$$\begin{pmatrix} 11 & 12 & 13 & 14 \\ 21 & 22 & 23 & 24 \\ 31 & 32 & 33 & 34 \end{pmatrix}$$

How do we lay out this matrix in memory?

$$\begin{pmatrix} 11 & 12 & 13 & 14 \\ 21 & 22 & 23 & 24 \\ 31 & 32 & 33 & 34 \end{pmatrix}$$

How do we lay out this matrix in memory?

Row-major order: where elements of each *row* are contiguous in memory:

11	12	13	14	21	22	23	24	31	32	33	34
----	----	----	----	----	----	----	----	----	----	----	----

with index function

$$(i,j) \mapsto i \times 4 + j$$

$$\begin{pmatrix} 11 & 12 & 13 & 14 \\ 21 & 22 & 23 & 24 \\ 31 & 32 & 33 & 34 \end{pmatrix}$$

How do we lay out this matrix in memory?

Row-major order: where elements of each *row* are contiguous in memory:

11	12	13	14	21	22	23	24	31	32	33	34
----	----	----	----	----	----	----	----	----	----	----	----

with index function

$$(i,j) \mapsto i \times 4 + j$$

Column-major order: where elements of each *column* are contiguous in memory:

11	21	31	12	22	32	13	23	33	14	24	34
----	----	----	----	----	----	----	----	----	----	----	----

with index function

$$(i,j) \mapsto j \times 3 + i$$

Two-dimensional index functions for $n \times m$ arrays

Row-major indexing

$$(i, j) \mapsto i \times m + j$$

Column-major indexing

$$(i, j) \mapsto j \times n + i$$

Intuition:

- Row-major indexing first *skips* i rows each comprising m elements, then jumps j elements into the row we reach.
- This is why n (the number of rows) is not used for row-major indexing.

Column-major has same intuition, but we skip size- n columns instead.

Two-dimensional arrays in C

$$\begin{pmatrix} 11 & 12 & 13 & 14 \\ 21 & 22 & 23 & 24 \\ 31 & 32 & 33 & 34 \end{pmatrix}$$

```
int *A =  
    malloc(3*4*sizeof(int));  
A[0] = 11; // first row  
A[1] = 12; // first row  
A[2] = 13; // first row  
A[3] = 14; // first row  
A[4] = 21; // second row  
...  
A[11] = 34;
```

```
int *A =  
    malloc(3*4*sizeof(int));  
A[0] = 11; // first col  
A[1] = 21; // first col  
A[2] = 31; // first col  
A[3] = 12; // second col  
A[4] = 22; // second col  
...  
A[11] = 34;
```

Index functions as C functions

```
int idx2_rowmajor(int n, int m, int i, int j) {  
    return i * m + j;  
}
```

```
int idx2_colmajor(int n, int m, int i, int j) {  
    return j * n + i;  
}
```

Useful if you get confused when writing index calculations by hand (I often do!)

Careful!

Consider indexing the 3×4 array from before with the expression

`A[idx2_rowmajor(n, m, 2, 5)]`.

- Trying to access index $(2, 5)$ —conceptually out of bounds.
- Index function translates to the flat index $2 \times 3 + 5 = 11$.
- This is *in-bounds* for the 12-element flat array we use to represent our matrix!
- Our program will not crash, but this is probably a bug.

Careful!

Consider indexing the 3×4 array from before with the expression

`A[idx2_rowmajor(n, m, 2, 5)].`

- Trying to access index (2, 5)—conceptually out of bounds.
- Index function translates to the flat index $2 \times 3 + 5 = 11$.
- This is *in-bounds* for the 12-element flat array we use to represent our matrix!
- Our program will not crash, but this is probably a bug.

```
int idx2_rowmajor(int n, int m, int i, int j) {  
    assert(i >= 0 && i < n);  
    assert(j >= 0 && j < m);  
    return i * m + j;  
}
```

Higher dimensions

For a d -dimensional row-major array of shape $n_0 \times \cdots \times n_{d-1}$, the index function where p is a d -dimensional index point is

$$p \mapsto \sum_{0 \leq i < d} p_i \times \prod_{i < j < d} n_j$$

where p_i gets the i th coordinate of p , and the product of an empty series is 1.

Intuition: p_i tells us how many “subarrays” of size $n_{i+1} \times \cdots \times n_{d-1}$ we need to skip.

Example: four-dimensional indexing

Suppose we have a row-major array of shape

$$n_0 \times n_1 \times n_2 \times n_3$$

and we wish to compute the flat index of element position

$$(p_0, p_1, p_2, p_3)$$

We then have to sum these terms where the *strides* s_i depend on the array size:

$$\begin{aligned} & p_0 \times s_0 \\ + & p_1 \times s_1 \\ + & p_2 \times s_2 \\ + & p_3 \times s_3 \end{aligned}$$

Example: four-dimensional indexing

Suppose we have a row-major array of shape

$$n_0 \times n_1 \times n_2 \times n_3$$

and we wish to compute the flat index of element position

$$(p_0, p_1, p_2, p_3)$$

We then have to sum these terms where the *strides* s_i depend on the array size:

$$\begin{aligned} & p_0 \times n_1 \times n_2 \times n_3 \\ + & p_1 \times n_2 \times n_3 \\ + & p_2 \times n_3 \\ + & p_3 \times 1 \end{aligned}$$

The stride s_i is the product $\prod_{j < i} n_j$ of the array size after dropping the first $i + 1$ dimensions.

Size passing

Since we represent arrays as the address of their first element, we must manually pass along the size when we call a function with an array.

```
double sumvec(int n, const double *vector) {  
    double sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += vector[i];  
    }  
    return sum;  
}
```

As usual: C will not protect us if we pass the wrong size. Be careful.

Slicing

- When using row-major order, the elements of each row are adjacent in memory.
- This allows us to perform efficient *slicing*, by taking the address of the first element in a row.

```
void sumrows(int n, int m,  
             const double *matrix, double *vector) {  
    for (int i = 0; i < n; i++) {  
        vector[i] = sumvec(m, &matrix[i*m]);  
    }  
}
```

Conclusions

- C's built-in arrays are suitable only for small arrays, typically of static size.
- Dynamic allocation can create single-dimensional dynamic arrays on the heap.
- We can represent multi-dimensional arrays as single-dimensional arrays, by specifying an index function.
- Careful when indexing these home-made arrays—the C language is not much help.

Arrays

Memory hierarchies

Cache organisation and operation

Cache performance

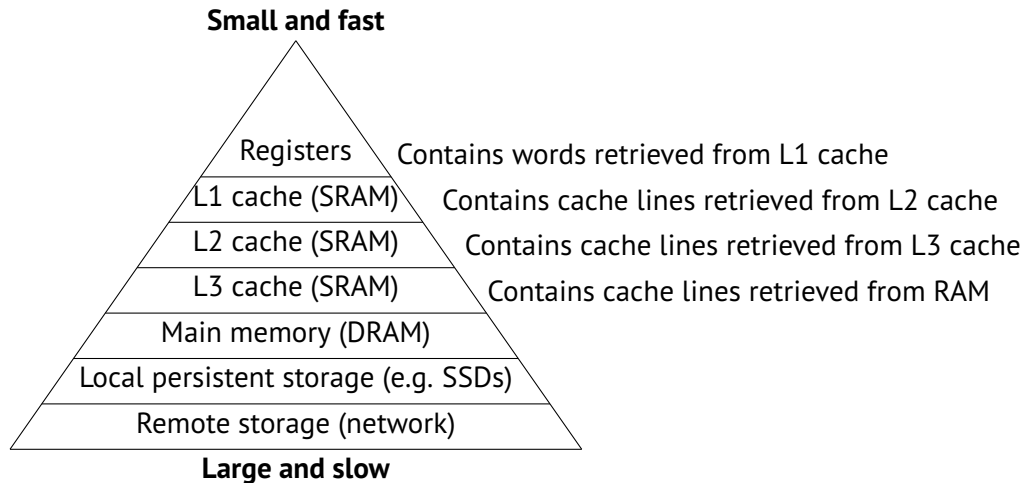
Memory hierarchies

Some fundamental and enduring properties of hardware and software

- Fast storage is expensive, has smaller capacity, and requires more power.
- There is a large gap between computational speed and memory speed.
- Well-written programs tend to exhibit good locality.

These properties suggest an approach for organising memory and other storage systems known as a *memory hierarchy*.

Example memory hierarchy



Definition of *cache*

A smaller, faster storage device that acts as a staging area for a subset of the data in a larger, slower device.

- **Fundamental idea of the memory hierarchy**

- ▶ The smaller and faster device at level k acts as a cache for the larger slower device at level $k + 1$.

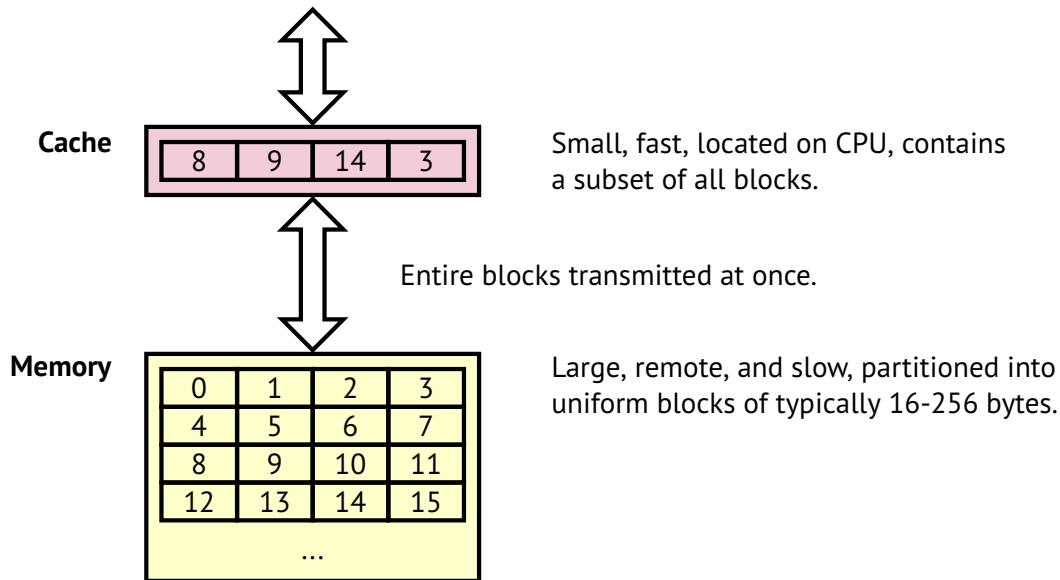
- **Why do they work?**

- ▶ Because of *locality*, most accesses tend to be towards the top of the hierarchy.

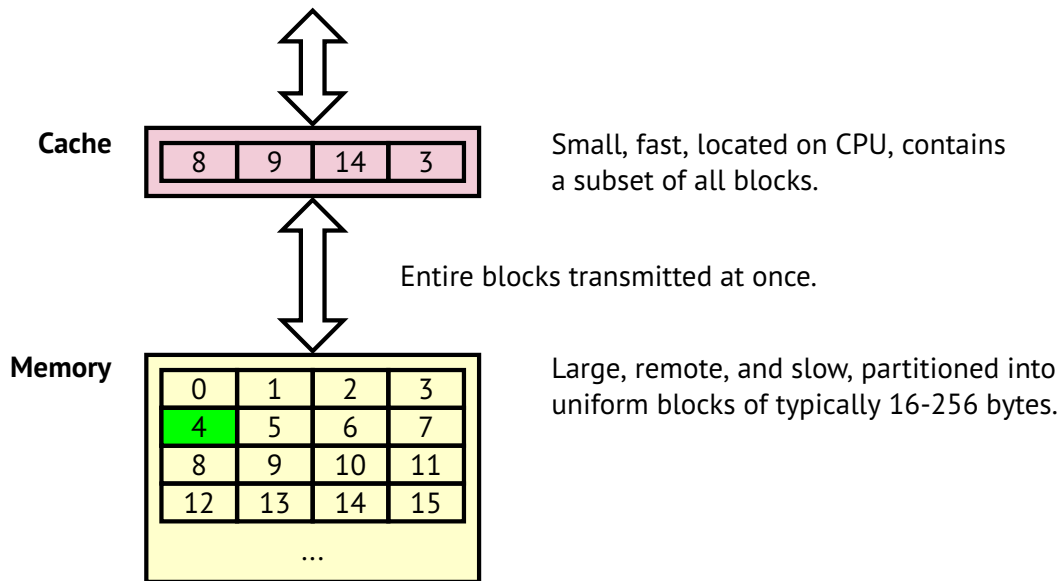
- **The ideal**

- ▶ A huge pool of storage that is as cheap as at the bottom of the hierarchy, but as fast as at the top of the hierarchy.

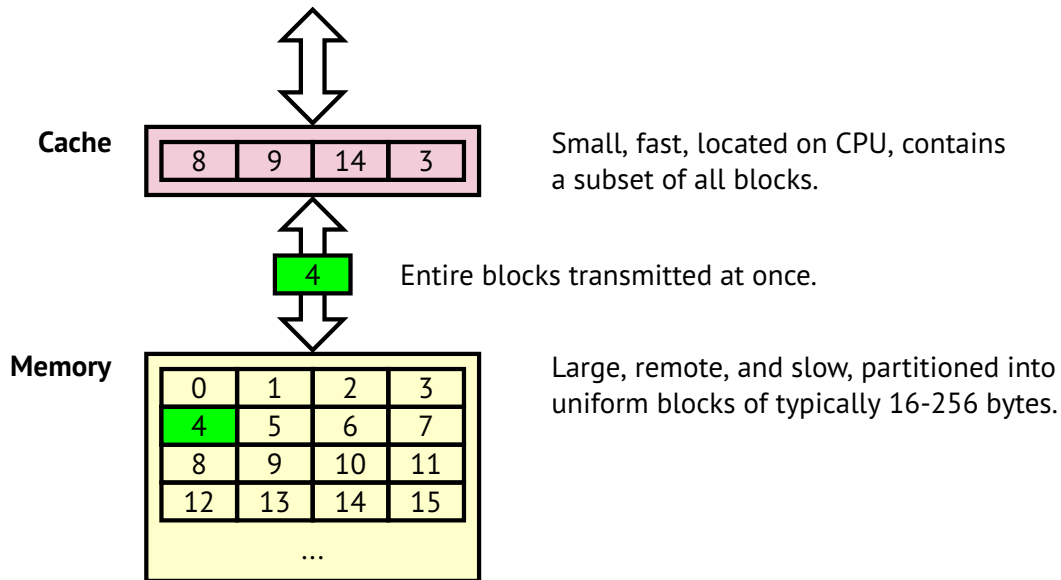
Cache overview



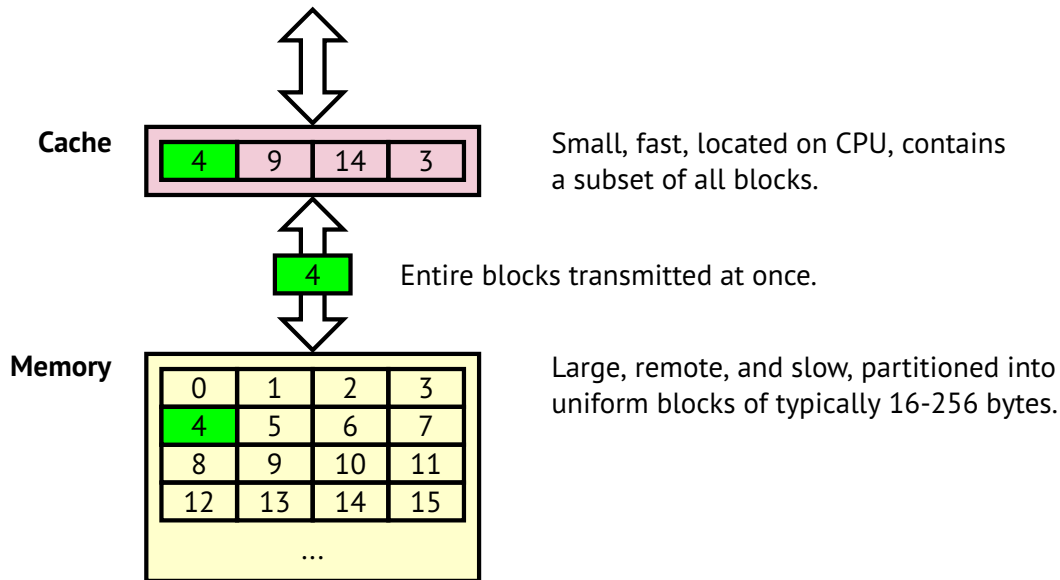
Cache overview



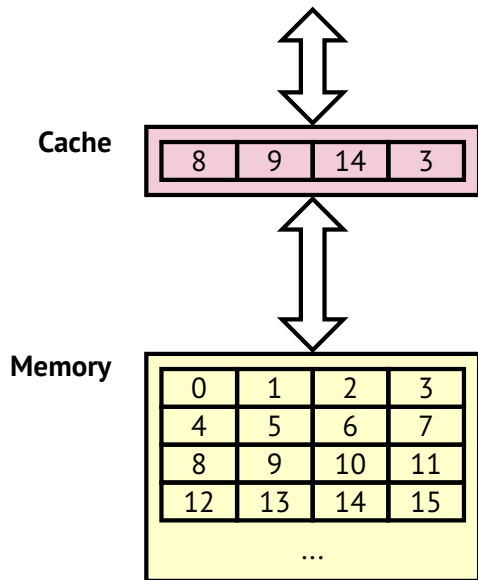
Cache overview



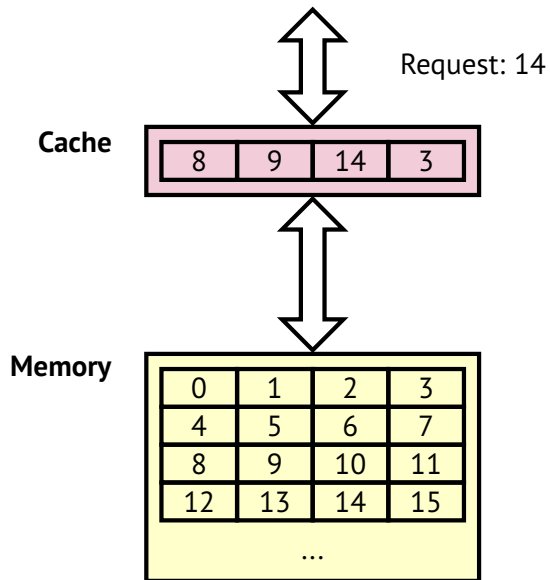
Cache overview



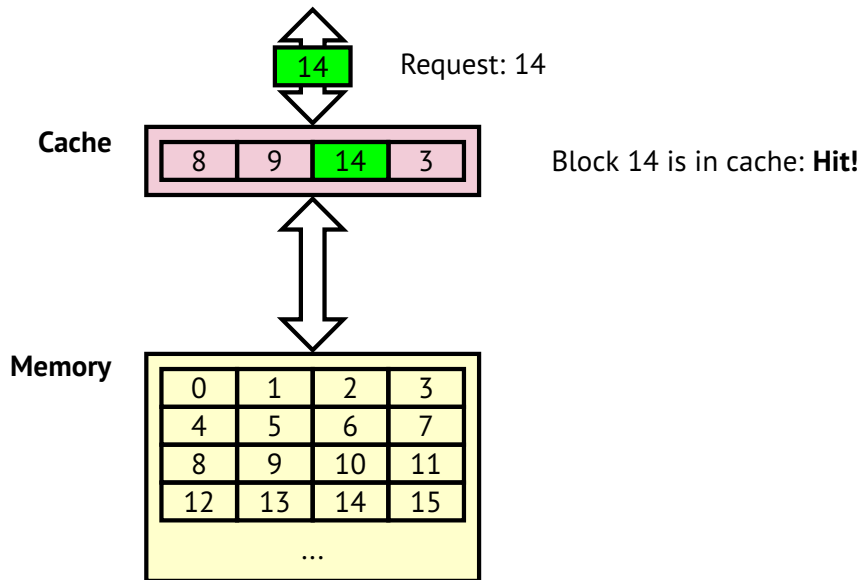
Example: cache hit



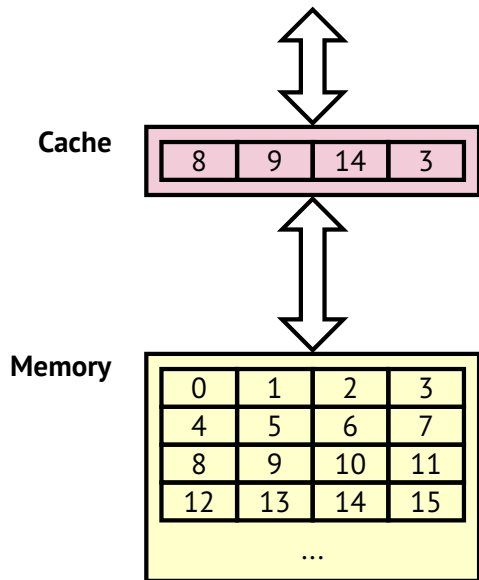
Example: cache hit



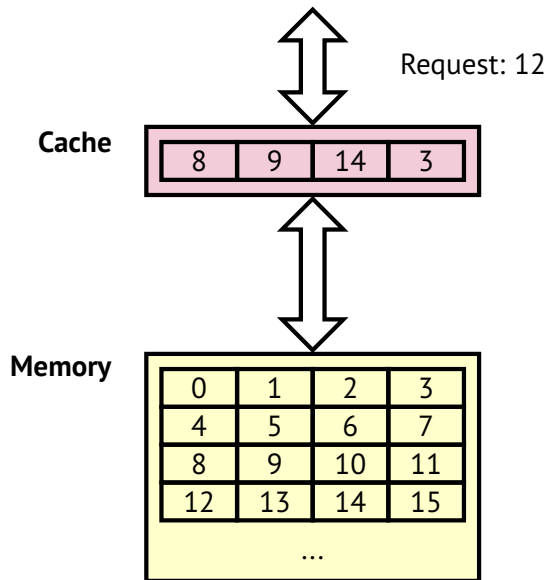
Example: cache hit



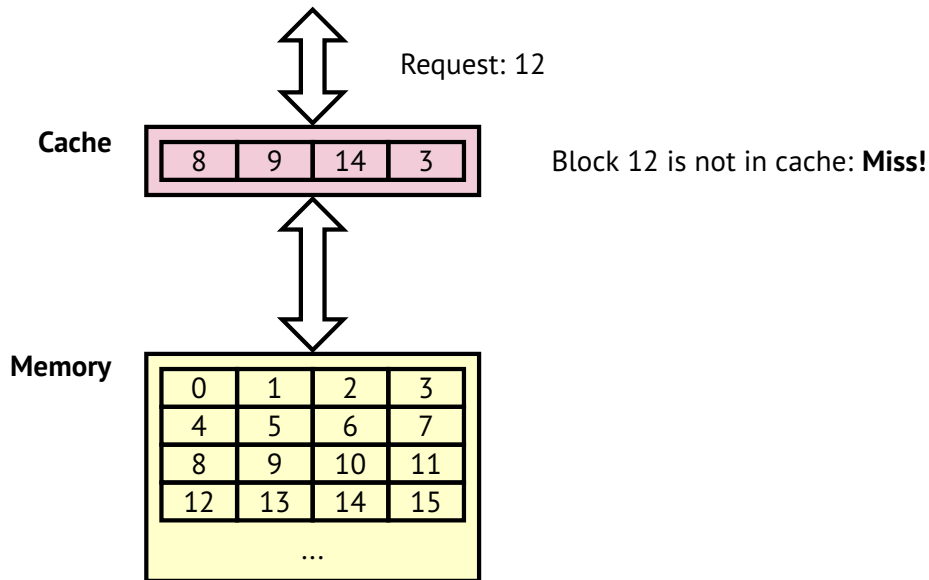
Example: cache miss



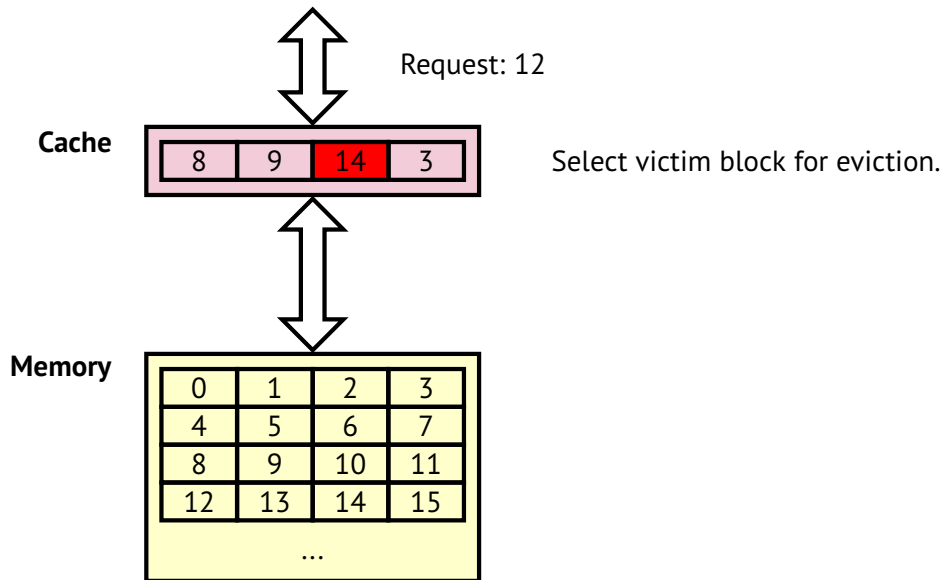
Example: cache miss



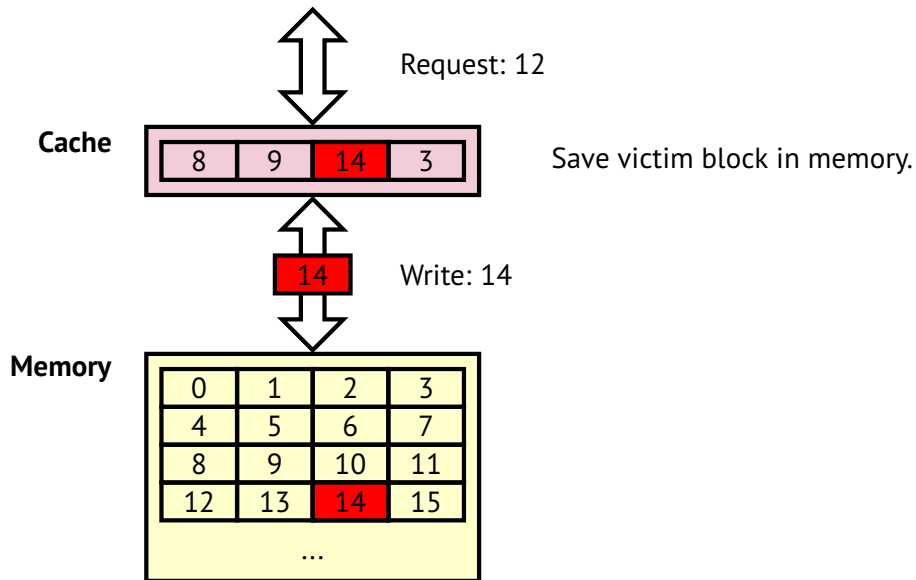
Example: cache miss



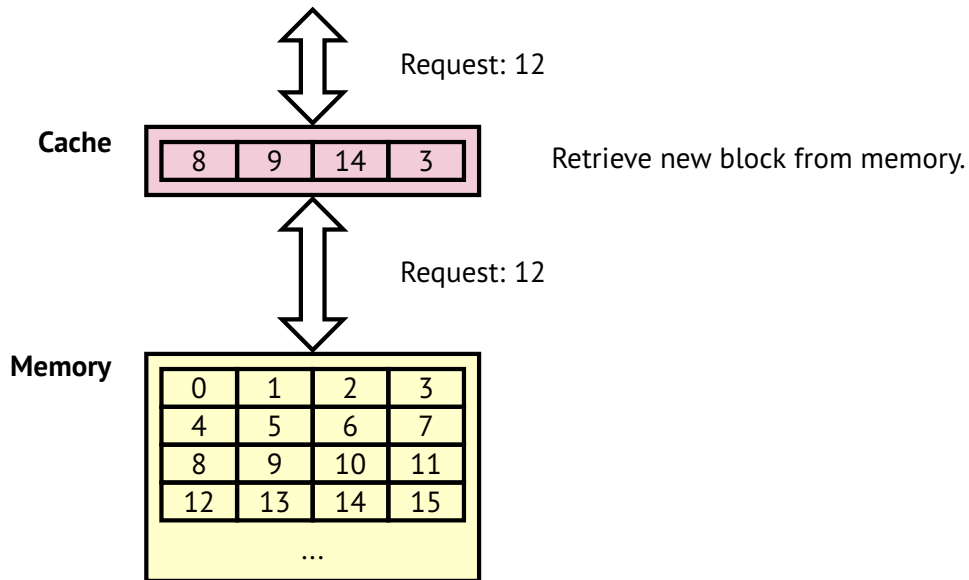
Example: cache miss



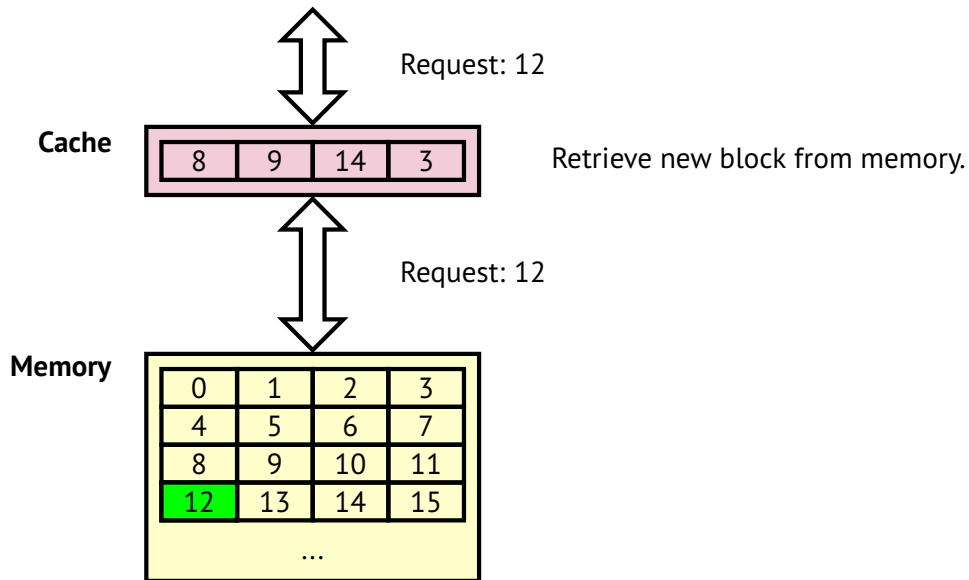
Example: cache miss



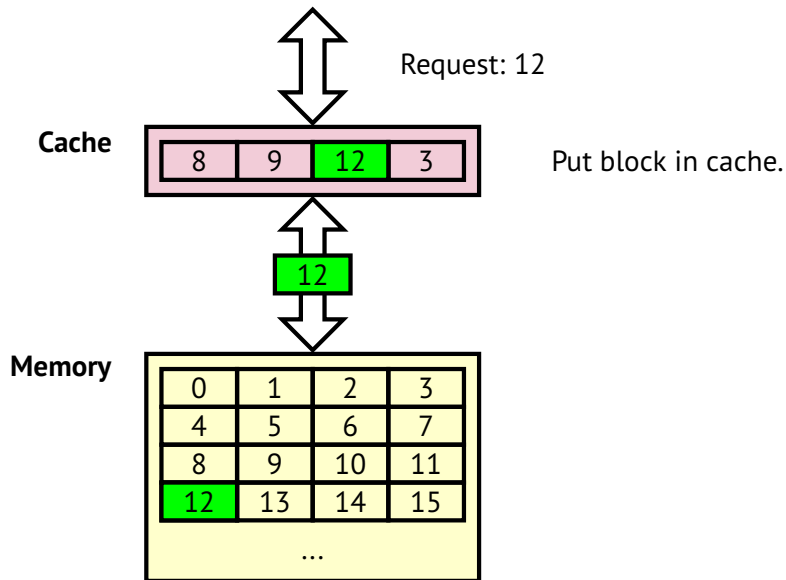
Example: cache miss



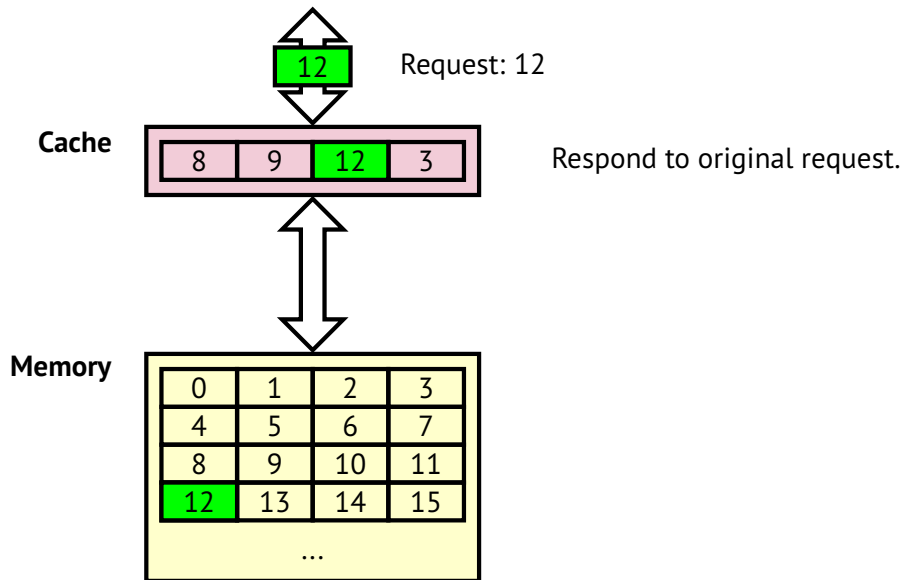
Example: cache miss



Example: cache miss



Example: cache miss



Types of cache misses

Cold/compulsory miss

- Occur when the cache is empty.
- Unavoidable when a program first starts.

Conflict miss

- Most caches limit blocks at level $k + 1$ to a small subset of the slots at level k .
 - ▶ **Example:** Block i can only be located in slot $i \bmod 4$.
- Causes conflicts when cache is large enough, but the blocks being accessed all map to the same slot.

Capacity miss

- Occurs when program *working set* exceeds size of cache.

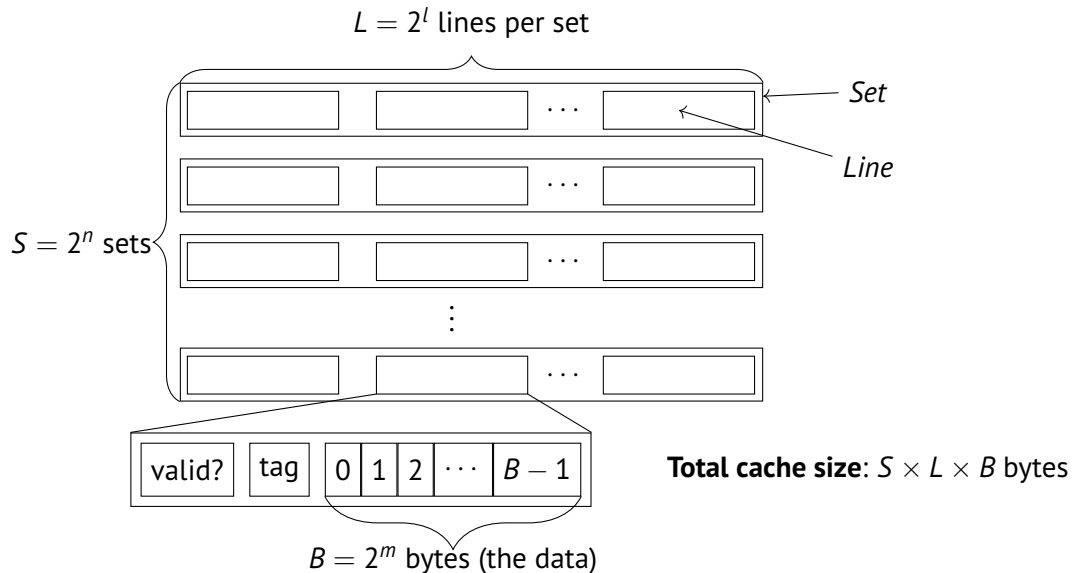
Arrays

Memory hierarchies

Cache organisation and operation

Cache performance

General structure of a cache for S , L , and B



Address structure

When $S = 2^n$, $B = 2^m$ we can easily split a w -bit address into *fields*, writing x_i for bit i .

$$\underbrace{x_{w-1} \cdots x_{m+n+1}}_{\text{tag}} \underbrace{x_{m+n} \cdots x_m}_{\text{set index}} \underbrace{x_{m-1} \cdots x_0}_{\text{block offset}}$$

Address structure

When $S = 2^n$, $B = 2^m$ we can easily split a w -bit address into *fields*, writing x_i for bit i .

$$\underbrace{x_{w-1} \cdots x_{m+n+1}}_{\text{tag}} \underbrace{x_{m+n} \cdots x_m}_{\text{set index}} \underbrace{x_{m-1} \cdots x_0}_{\text{block offset}}$$

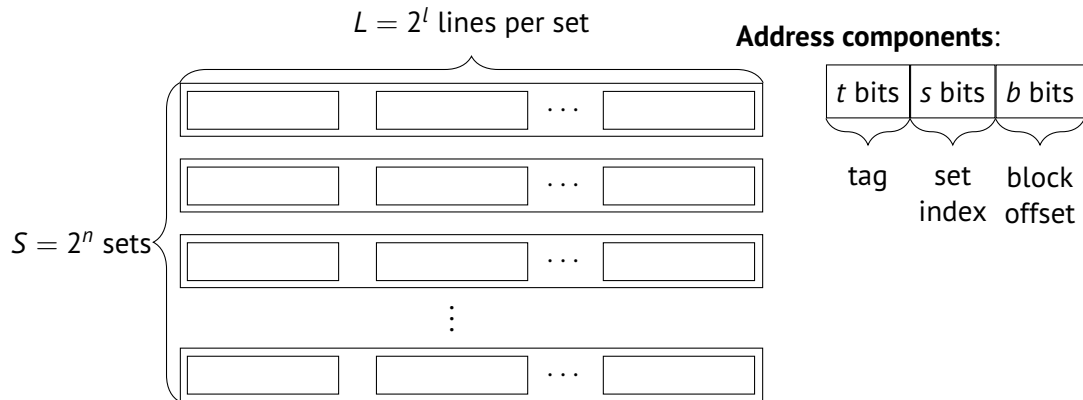
Example

Consider an 8-bit address with $m = 2$, $s = 3$.

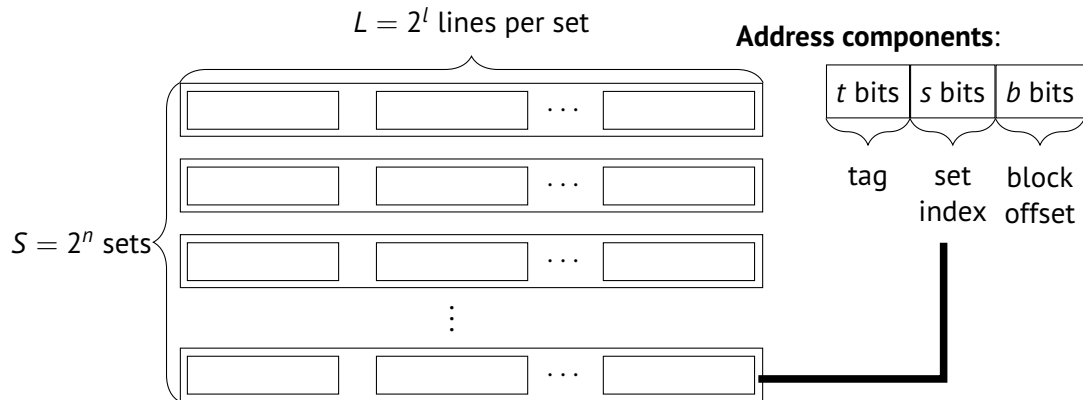
$$\underbrace{x_7 x_6 x_5}_{\text{tag}} \underbrace{x_4 x_3 x_2}_{\text{index}} \underbrace{x_1 x_0}_{\text{offset}}$$

We look up an address in the cache by splitting the address into fields and looking up and checking based on their values.

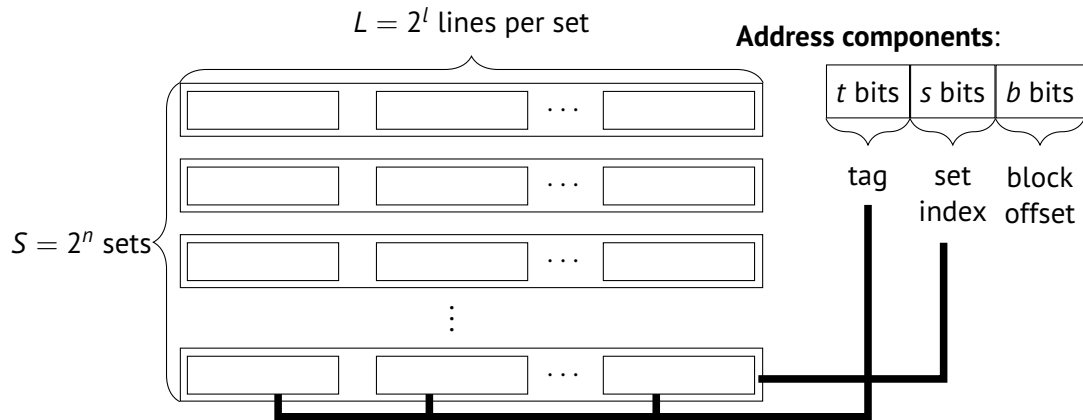
Cache lookup



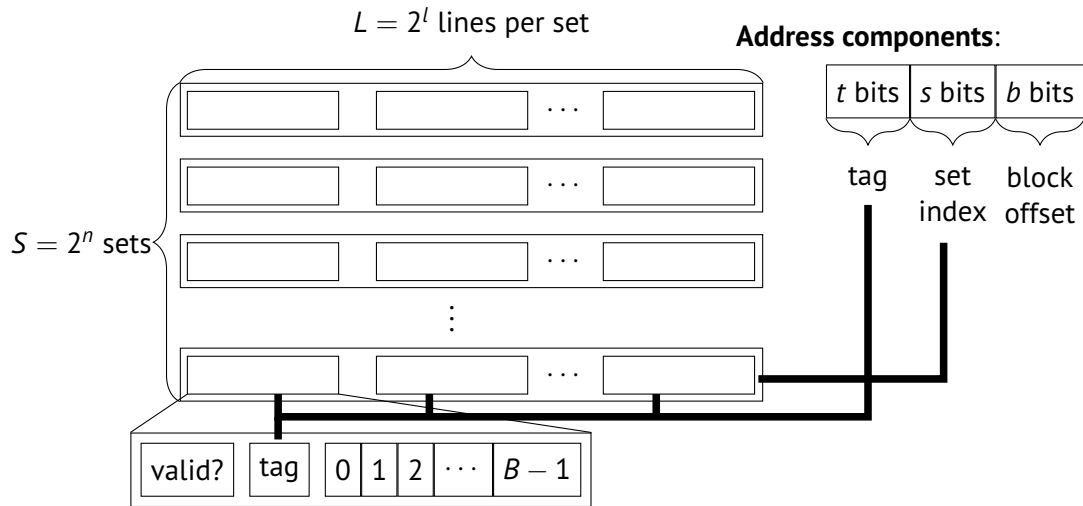
Cache lookup



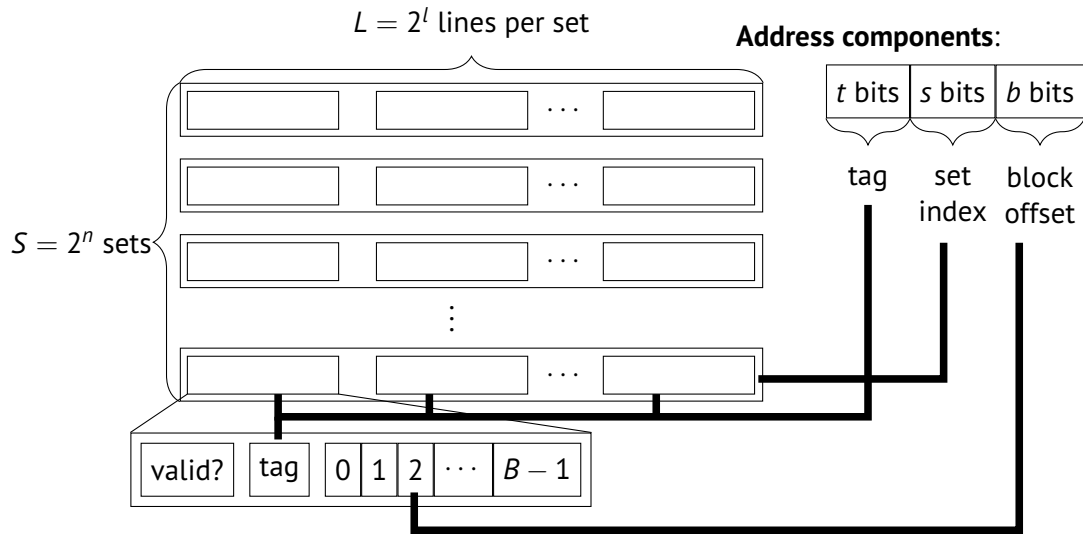
Cache lookup



Cache lookup



Cache lookup



Example: Direct-Mapped ($L = 1$), with 4 sets and 8-byte blocks ($B = 8$)

Suppose 10-bit addresses, so $b = 3, s = 2, t = 6$. **Note:** one line per set.



Example: Direct-Mapped ($L = 1$), with 4 sets and 8-byte blocks ($B = 8$)

Suppose 10-bit addresses, so $b = 3, s = 2, t = 6$. **Note:** one line per set.

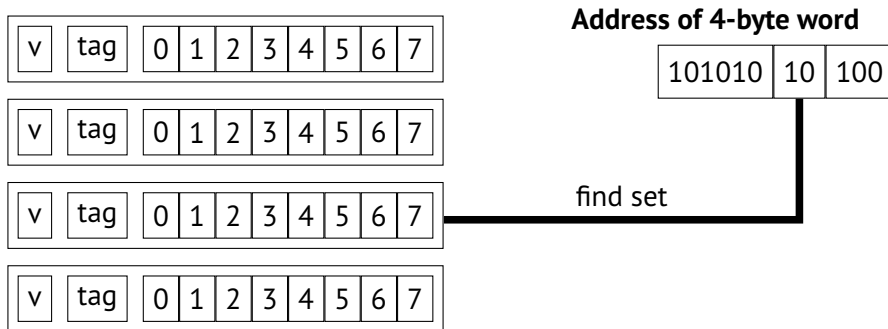


Address of 4-byte word

101010	10	100
--------	----	-----

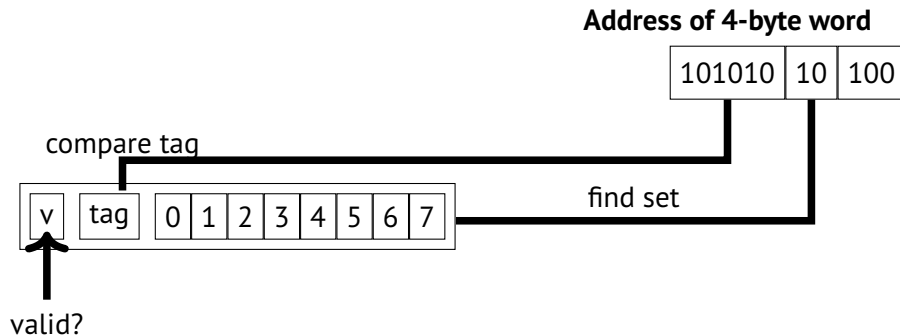
Example: Direct-Mapped ($L = 1$), with 4 sets and 8-byte blocks ($B = 8$)

Suppose 10-bit addresses, so $b = 3, s = 2, t = 6$. **Note:** one line per set.



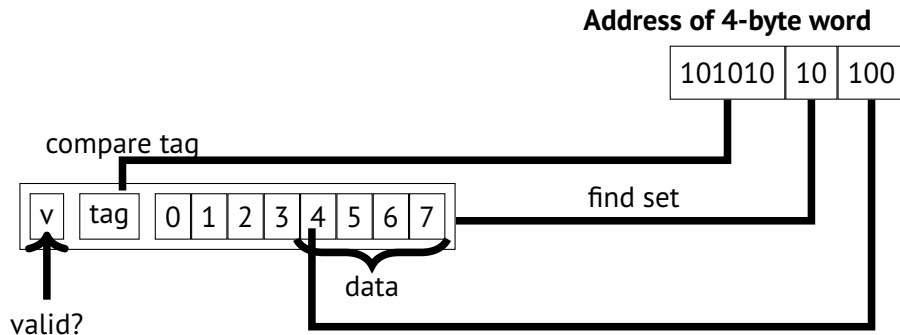
Example: Direct-Mapped ($L = 1$), with 4 sets and 8-byte blocks ($B = 8$)

Suppose 10-bit addresses, so $b = 3, s = 2, t = 6$. **Note:** one line per set.



Example: Direct-Mapped ($L = 1$), with 4 sets and 8-byte blocks ($B = 8$)

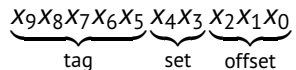
Suppose 10-bit addresses, so $b = 3, s = 2, t = 6$. **Note:** one line per set.



Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



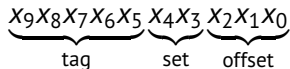
Simulation, reading single bytes from address

	Valid	Tag	Block
Set 0	0		
Set 1	0		
Set 2	0		
Set 3	0		

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

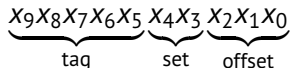
■ 00000 00 000

	Valid	Tag	Block
Set 0	0		
Set 1	0		
Set 2	0		
Set 3	0		

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

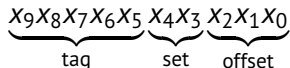
■ 00000 00 000 **Miss**

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	0		

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

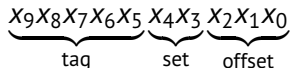
- 00000 00 000 **Miss**
- 00000 00 001

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	0		

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

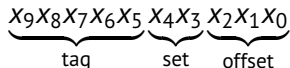
- 00000 00 000 **Miss**
- 00000 00 001 **Hit**

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	0		

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

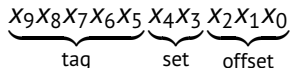
- 00000 00 000 **Miss**
- 00000 00 001 **Hit**
- 01000 11 100

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	0		

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

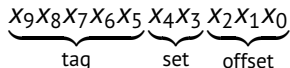
- 00000 00 000 **Miss**
- 00000 00 001 **Hit**
- 01000 11 100 **Miss**

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	1	01000	Mem[280-287]

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

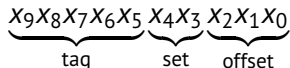
- 00000 00 000 **Miss**
- 00000 00 001 **Hit**
- 01000 11 100 **Miss**
- 00001 00 000

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	1	01000	Mem[280-287]

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

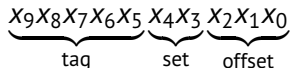
- 00000 00 000 **Miss**
- 00000 00 001 **Hit**
- 01000 11 100 **Miss**
- 00001 00 000 **Miss**

	Valid	Tag	Block
Set 0	1	00001	Mem[32-39]
Set 1	0		
Set 2	0		
Set 3	1	01000	Mem[280-287]

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

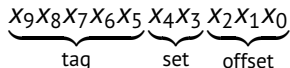
- 00000 00 000 **Miss**
- 00000 00 001 **Hit**
- 01000 11 100 **Miss**
- 00001 00 000 **Miss**
- 00000 00 000

	Valid	Tag	Block
Set 0	1	00001	Mem[32-39]
Set 1	0		
Set 2	0		
Set 3	1	01000	Mem[280-287]

Simulation of Direct-Mapped Cache

Characteristics

10-bit addresses, $B = 8$, $S = 4$, $L = 1$.



Simulation, reading single bytes from address

- 00000 00 000 **Miss**
- 00000 00 001 **Hit**
- 01000 11 100 **Miss**
- 00001 00 000 **Miss**
- 00000 00 000 **Miss**

	Valid	Tag	Block
Set 0	1	00000	Mem[0-7]
Set 1	0		
Set 2	0		
Set 3	1	01000	Mem[280-287]

Example: Set-associative ($L = 2$), with 4 sets and 8-byte blocks ($B = 8$)



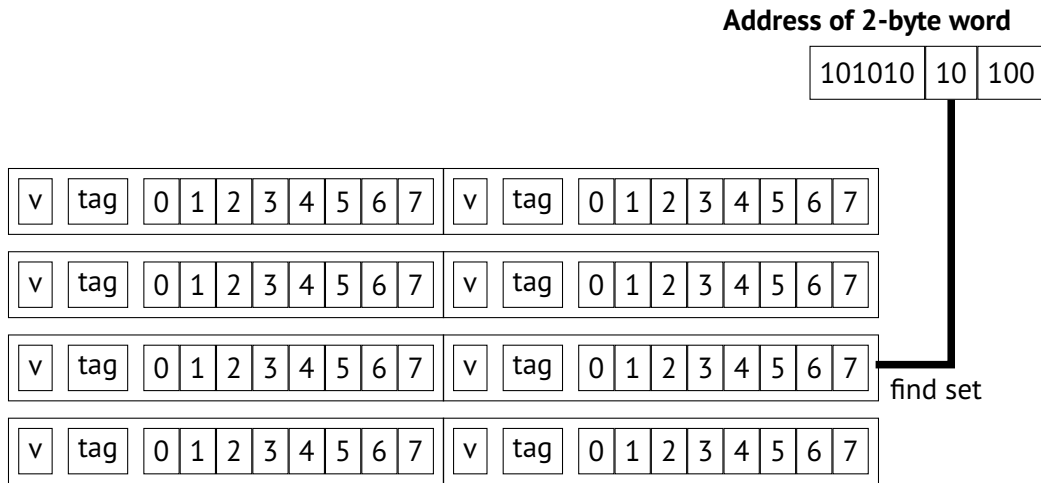
Example: Set-associative ($L = 2$), with 4 sets and 8-byte blocks ($B = 8$)

Address of 2-byte word

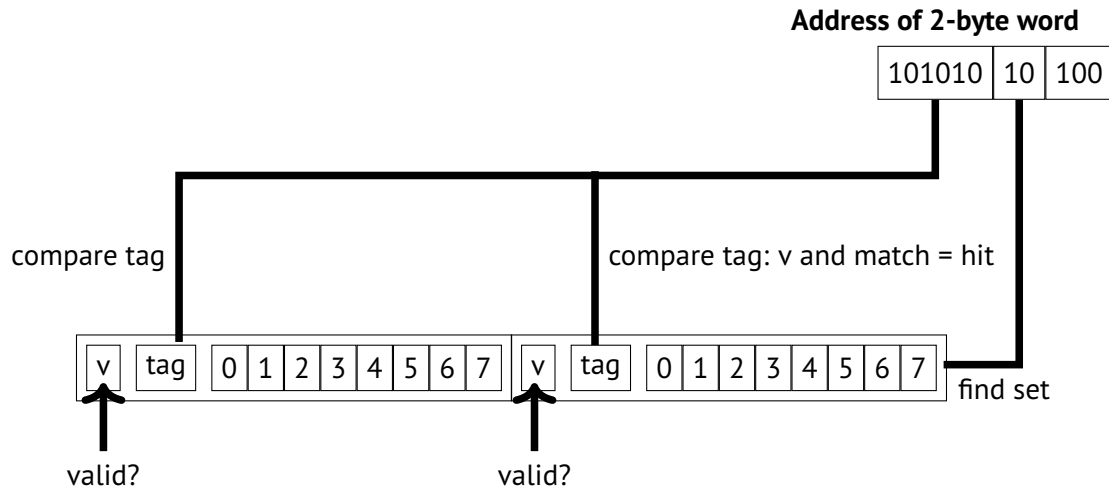
101010	10	100
--------	----	-----



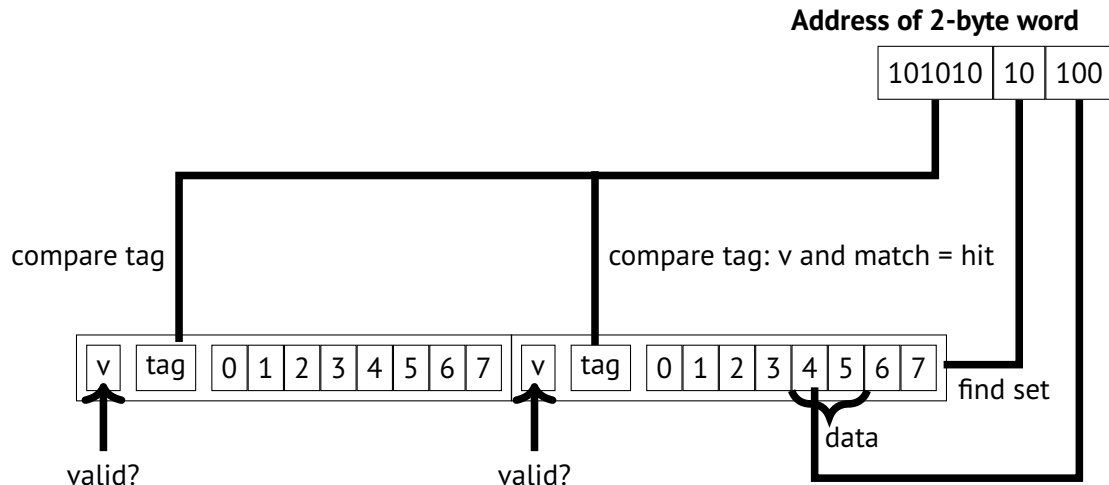
Example: Set-associative ($L = 2$), with 4 sets and 8-byte blocks ($B = 8$)



Example: Set-associative ($L = 2$), with 4 sets and 8-byte blocks ($B = 8$)



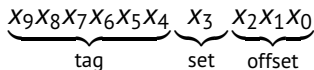
Example: Set-associative ($L = 2$), with 4 sets and 8-byte blocks ($B = 8$)



Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



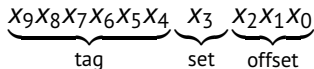
Simulation, reading single bytes from address

	Valid	Tag	Block
Set 0	0		
	0		
Set 1	0		
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

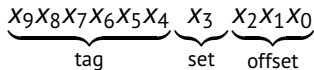
■ 000000 0 000

	Valid	Tag	Block
Set 0	0		
	0		
Set 1	0		
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

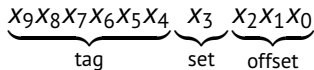
■ 000000 0 000 **Miss**

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	0		
Set 1	0		
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

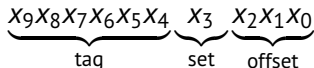
- 000000 0 000 **Miss**
- 000000 0 001

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	0		
Set 1	0		
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

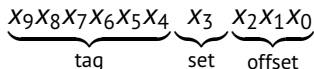
- 000000 0 000 **Miss**
- 000000 0 001 **Hit**

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	0		
Set 1	0		
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

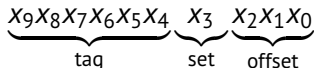
- 000000 0 000 **Miss**
- 000000 0 001 **Hit**
- 010001 1 100

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	0		
Set 1	0		
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

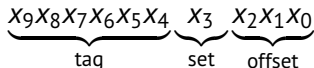
- 000000 0 000 **Miss**
- 000000 0 001 **Hit**
- 010001 1 100 **Miss**

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	0		
Set 1	1	010001	Mem[280-287]
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

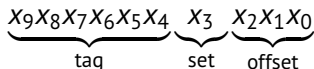
- 000000 0 000 **Miss**
- 000000 0 001 **Hit**
- 010001 1 100 **Miss**
- 000010 0 000

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	0		
Set 1	1	010001	Mem[280-287]
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

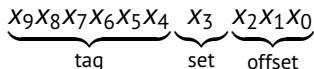
- 000000 0 000 **Miss**
- 000000 0 001 **Hit**
- 010001 1 100 **Miss**
- 000010 0 000 **Miss**

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	1	000010	Mem[32-39]
Set 1	1	010001	Mem[280-287]
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

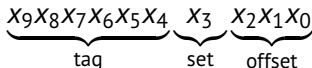
- 000000 0 000 **Miss**
- 000000 0 001 **Hit**
- 010001 1 100 **Miss**
- 000010 0 000 **Miss**
- 000000 0 000

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	1	000010	Mem[32-39]
Set 1	1	010001	Mem[280-287]
	0		

Simulation of 2-way set-associative cache

Characteristics

10-bit addresses, $B = 8$, $S = 1$, $L = 2$.



Simulation, reading single bytes from address

- 000000 0 000 **Miss**
- 000000 0 001 **Hit**
- 010001 1 100 **Miss**
- 000010 0 000 **Miss**
- 000000 0 000 **Hit**

	Valid	Tag	Block
Set 0	1	000000	Mem[0-7]
	1	000010	Mem[32-39]
Set 1	1	010001	Mem[280-287]
	0		

What about writes?

Multiple copies of data exist

- L1, L2, L3 caches, main memory, disk, backup in the cloud...

What do we do on a write hit?

Write-through: writing immediately to the next level of the hierarchy.

Write-back: defer write until the cache block is evicted.

- Needs a *dirty bit* indicating whether block changed since it was loaded.

What do we do on a write miss?

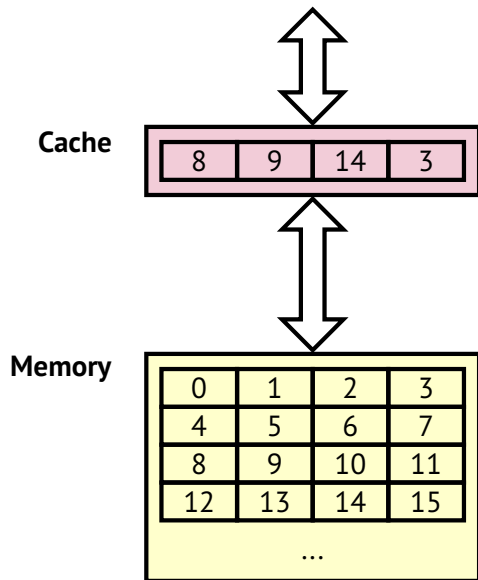
Write-allocate: load block into cache and update there.

- Good if more writes follow.

No-write-allocate: write straight to next level, do not load into cache.

CPU caches are typically write-back and write-allocate.

Cache



Arrays

Memory hierarchies

Cache organisation and operation

Cache performance

A real cache

- Use `sudo dmidecode -t cache` or `lscpu` on Linux to see hardware details, including CPU cache specs.
- On an Ryzen 1700X, cache blocks are 64 bytes each, and
 - ▶ **L1:** 96KiB, 8-way set-associative, split into 32KiB for data (L1d) and 64KiB for instructions (L1i).
 - ▶ **L2:** 512KiB, 8-way set-associative.
 - ▶ **L3:** 16MiB, 8-way set associative.
- **But:** Each of the 8 cores have their own L1 and L2 caches, but L3 cache is shared.
 - ▶ For those who think caches are very fascinating, read up on *cache coherency protocols* and *false sharing*.

Cache performance metrics

Miss rate

- Fraction of memory references not found in cache ($\text{misses} \div \text{accesses}$).
- Typical numbers:
 - ▶ 3-10% for L1
 - ▶ Can be very small ($< 1\%$) for L2.

Hit time

- Time to deliver a cache block to the processor.
 - ▶ Includes time to determine whether a hit (checking tag).
- Typical numbers
 - ▶ L1: 4 clock cycles.
 - ▶ L2: 10 clock cycles.
 - ▶ L3: 40-75 cycles (depends on sharing).

Miss penalty

- Additional time needed because of a miss.
- Often over 100 cycles for main DRAM memory, historically increasing.

Conceptualising those numbers

- **Huge difference between a hit and a miss!**

- ▶ Could be $100 \times$ between L1 and main memory.

- **99% hit rate may be twice as good as 97%**

Cache hit time: 1 cycle

Miss penalty: 100 cycles

Average access time:

- ▶ 97% hits: $1 \text{ cycle} + 0.03 \times 100 \text{ cycles} = 4 \text{ cycles}$

- ▶ 99% hits: $1 \text{ cycle} + 0.01 \times 100 \text{ cycles} = 2 \text{ cycles}$

- **This is why we use *miss rate* instead of *hit rate*.**

Writing cache friendly code

- **Make the common case go fast.**
 - ▶ Focus on inner loops.
 - ▶ Optimising non-loopy code is almost never worth it.

Writing cache friendly code

- **Make the common case go fast.**
 - ▶ Focus on inner loops.
 - ▶ Optimising non-loopy code is almost never worth it.
- **Ensure good locality.**
 - ▶ Avoid chasing ad-hoc pointers.
 - ▶ If you `malloc()` a million tiny blocks of memory, there is no guarantee they will be anywhere near each other.
 - ▶ Linked lists are *terrible*.

Writing cache friendly code

- **Make the common case go fast.**
 - ▶ Focus on inner loops.
 - ▶ Optimising non-loopy code is almost never worth it.
- **Ensure good locality.**
 - ▶ Avoid chasing ad-hoc pointers.
 - ▶ If you `malloc()` a million tiny blocks of memory, there is no guarantee they will be anywhere near each other.
 - ▶ Linked lists are *terrible*.
- **Minimise *footprint*.**
 - ▶ E.g. if all the data you work on fits in L3 cache, then you will never see an L3 cache miss after the initial ones.

Example: matrix multiplication

```
for (int i=0; i<n; i++) {  
    for (int j=0; j<n; j++) {  
        double sum = 0.0;  
        for (int k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

- Multiply n -by- n matrices in row-major order.
- Each element is a `double`
- $O(n^3)$ total operations.
- n values summed per destination.

Miss rate analysis

Assume

- Cache block size $B = 64\text{bytes}$
 - ▶ Enough for 8 doubles.
- n is very large—approximate $1/n$ as 0.
- Cache is not even big enough to hold a single row or column.

Method

- Look at access pattern of inner loop.

Miss rate analysis

- Assume**
- Cache block size $B = 64\text{bytes}$
 - ▶ Enough for 8 doubles.
 - n is very large—approximate $1/n$ as 0.
 - Cache is not even big enough to hold a single row or column.
- Method**
- Look at access pattern of inner loop.

When accessing successive elements miss rate is
 $\text{sizeof}(\text{double})/B = 0.125$.

```
for (int k=0; k<n; k++)  
    sum += a[0][k]
```

Miss rate analysis

Assume

- Cache block size $B = 64\text{bytes}$
 - Enough for 8 doubles.
- n is very large—approximate $1/n$ as 0.
- Cache is not even big enough to hold a single row or column.

Method

- Look at access pattern of inner loop.

When accessing successive elements miss rate is $\text{sizeof}(\text{double})/B = 0.125$.

```
for (int k=0; k<n; k++)  
    sum += a[0][k]
```

When accessing distant elements miss rate is 1.

```
for (int k=0; k<n; k++)  
    sum += a[k][0]
```

Matrix multiplication (ijk)

```
for (int i=0; i<n; i++) {  
    for (int j=0; j<n; j++) {  
        double sum = 0.0;  
        for (int k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

Misses per innermost loop iteration

a

b

c

Matrix multiplication (ijk)

```
for (int i=0; i<n; i++) {  
    for (int j=0; j<n; j++) {  
        double sum = 0.0;  
        for (int k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

Misses per innermost loop iteration

a	b	c
0.125	1	0

Matrix multiplication (jik)

```
for (int j=0; j<n; j++) {  
    for (int i=0; i<n; i++) {  
        double sum = 0.0;  
        for (int k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum  
    }  
}
```

Misses per innermost loop iteration

a

b

c

Matrix multiplication (jik)

```
for (int j=0; j<n; j++) {  
    for (int i=0; i<n; i++) {  
        double sum = 0.0;  
        for (int k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum  
    }  
}
```

Misses per innermost loop iteration

a	b	c
0.125	1	0

Matrix multiplication (kij)

```
for (int k=0; k<n; k++) {  
    for (int i=0; i<n; i++) {  
        double r = a[i][k];  
        for (int j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

Misses per innermost loop iteration

a

b

c

Matrix multiplication (kij)

```
for (int k=0; k<n; k++) {  
    for (int i=0; i<n; i++) {  
        double r = a[i][k];  
        for (int j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

Misses per innermost loop iteration

a	b	c
0	0.125	0.125

Matrix multiplication (ikj)

```
for (int i=0; i<n; i++) {  
    for (int k=0; k<n; k++) {  
        double r = a[i][k];  
        for (int j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

Misses per innermost loop iteration

a

b

c

Matrix multiplication (ikj)

```
for (int i=0; i<n; i++) {  
    for (int k=0; k<n; k++) {  
        double r = a[i][k];  
        for (int j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

Misses per innermost loop iteration

a	b	c
0	0.125	0.125

Matrix multiplication (jki)

```
for (int j=0; j<n; j++) {  
    for (int k=0; k<n; k++) {  
        double r = b[k][j];  
        for (int i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

Misses per innermost loop iteration

a

b

c

Matrix multiplication (jki)

```
for (int j=0; j<n; j++) {  
    for (int k=0; k<n; k++) {  
        double r = b[k][j];  
        for (int i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

Misses per innermost loop iteration

a	b	c
1	0	1

Matrix multiplication (kji)

```
for (int k=0; k<n; k++) {  
    for (int j=0; j<n; j++) {  
        double r = b[k][j];  
        for (int i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

Misses per innermost loop iteration

a

b

c

Matrix multiplication (kji)

```
for (int k=0; k<n; k++) {  
    for (int j=0; j<n; j++) {  
        double r = b[k][j];  
        for (int i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

Misses per innermost loop iteration

a	b	c
1	0	1

Summary of variants

```
for (int i=0; i<n; i++) {  
    for (int j=0; j<n; j++) {  
        double sum = 0.0;  
        for (int k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

```
for (int k=0; k<n; k++) {  
    for (int i=0; i<n; i++) {  
        double r = a[i][k];  
        for (int j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

```
for (j=0; j<n; j++) {  
    for (k=0; k<n; k++) {  
        double r = b[k][j];  
        for (int i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

ijk and jik (inner loop):

- 2 loads, 0 stores
- 1.125 misses/iter

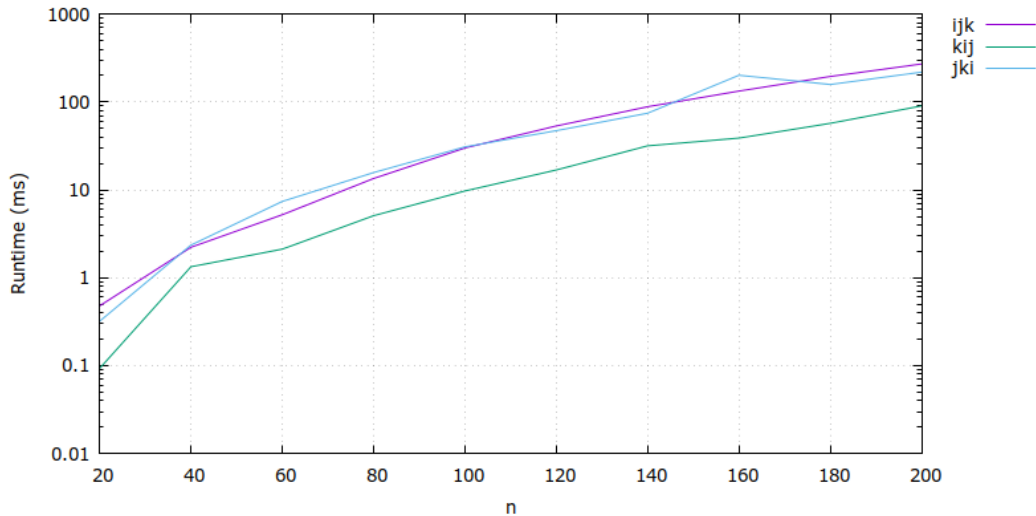
kij and ikj (inner loop):

- 2 loads, 1 store
- 0.25 misses/iter

jki and kji (inner loop):

- 2 loads, 1 store
- 2 misses/iter

On a real machine



Summary

- Memory hierarchies are part of all nontrivial systems.
- Cache misses have dramatic impact on performance.
- Significant speedup can be achieved just by permuting loops.